

DATA TYPES

What are Data Types in Python?

A Data Type defines:

- What kind of value is stored
- How memory is allocated
- Whether the value can be modified or not
- How Python internally manages the data

Python automatically identifies the datatype when we assign a value.

Example:

Python

```
x = 10  
name = "Rohit"  
price = 99.5
```

Python internally understands:

- 10 → int
- "Rohit" → str
- 99.5 → float

Classification of Python Data Types

Python Data Types are mainly divided into:

1. Numeric Types
2. Sequence Types
3. Set Types
4. Mapping Type
5. Boolean Type
6. None Type

1. NUMERIC TYPES

Numeric datatypes are used to store numerical values.

Memory Allocation:

- Memory stores actual numeric values directly
- Optimized for mathematical calculations
- Stored internally in binary format

Types:

1. int
2. float
3. complex

a. int – Integer Type

Stores whole numbers without decimal values.

Examples:

Python

```
age = 21
product_quantity = 5
order_id = 10987432
```

Characteristics:

- Stores positive and negative whole numbers
- No decimal point
- Unlimited size in Python

Example:

Python

```
x = 100
```

Mutable or Immutable?

- Immutable

Explanation:

Once an integer object is created, Python does not modify that object internally. If the value changes, Python creates a new object in memory.

Example:

Python

```
x = 10  
x = 20
```

b. float – Floating Point Type

Stores decimal values.

Examples:

Python

```
price = 749.99  
discount = 15.5  
rating = 4.3
```

Characteristics:

- Supports decimal numbers
- Used in price calculations and measurements
- Requires more memory than integers

Memory Allocation:

- Stored using floating-point representation internally

Example:

Python

```
temperature = 36.7
```

Mutable or Immutable?

- Immutable

c. complex – Complex Number Type

Stores real and imaginary numbers.

Syntax:

Python

```
z = 5 + 2j
```

Where:

- 5 → real part
- 2j → imaginary part

Used In:

- Scientific calculations
- Signal processing
- Advanced mathematics

Example:

Python

```
a = 3 + 4j
```

Mutable or Immutable?

- Immutable

2. SEQUENCE TYPES

Sequence datatypes store multiple values in a specific order.

Why are they called Sequence Types?

Because:

- Elements are stored sequentially
- Each element has an index position
- Order is maintained

Memory Allocation:

- Data is arranged one after another
- Supports indexing and traversal

Example:

Python

```
Index:    0      1      2  
List : [ "A", "B", "C" ]
```

Types:

1. str
2. list
3. tuple

a. str – String Type

Stores textual data.

Examples:

Python

```
name = "Rohit Sharma"  
city = "Hyderabad"
```

Characteristics:

- Collection of characters
- Ordered datatype
- Supports indexing

Example:

Python

```
text = "Python"
```

Memory Allocation:

- Characters are stored in sequence internally

Mutable or Immutable?

- Immutable

Explanation:

Characters inside a string cannot be modified directly.

Example:

```
Python
name = "Python"

# Invalid
name[0] = "J"
```

b. list – List Type

Stores multiple values in ordered form.

Example:

```
Python
cart_items = ["Shoes", "Watch", "Bag"]
```

Characteristics:

- Ordered
- Indexed
- Allows duplicates
- Dynamic size

Memory Allocation:

- Elements are stored sequentially
- Python internally stores references to objects
- Size can increase dynamically

Example:

```
Python
numbers = [10, 20, 30]
```

Mutable or Immutable?

- Mutable

Meaning:

Values inside the list can be modified after creation.

Example:

```
Python
numbers = [10, 20, 30]

numbers[1] = 100
```

Why List is Mutable?

Because Python allows modification of internal memory locations of list elements.

c. tuple – Tuple Type

Stores ordered fixed data.

Example:

```
Python
dimensions = (10, 20, 30)
```

Characteristics:

- Ordered
- Indexed
- Allows duplicates
- Faster than lists

Memory Allocation:

- Fixed-size memory allocation
- Optimized for read-only operations

Mutable or Immutable?

- Immutable

Example:

```
Python
data = (1, 2, 3)

# Invalid
data[0] = 100
```

Why is Tuple Immutable?

Because Python protects tuple memory from modification after creation.

Advantages:

- Faster execution
- Less memory usage
- More secure for constant data

3. SET TYPES

Set datatypes store unique values.

Characteristics:

- No duplicate values
- Unordered collection
- No indexing

Memory Allocation:

- Uses hashing internally
- Faster searching
- Duplicate values automatically removed

Types:

1. set
2. frozenset

a. set – Set Type

Example:

```
Python
colors = {"Red", "Blue", "Green"}
```

Duplicate Removal:

```
Python
brands = {"Nike", "Puma", "Nike"}
```


Output internally becomes:

```
Python  
{ "Nike", "Puma" }
```

Why?

Because sets only store unique values.

Mutable or Immutable?

- Mutable

Example:

```
Python  
sizes = { "S", "M" }
```

New values can be added later.

b. frozenset – Frozen Set

Immutable version of set.

Example:

```
Python  
tags = frozenset([ "Sale", "Trending" ])
```

Characteristics:

- No modification allowed
- Fixed unique collection

Mutable or Immutable?

- Immutable

Explanation:

Once created, values inside the frozenset cannot be changed.

4. MAPPING TYPE

dict – Dictionary

The dictionary stores data in key-value pairs.

Example:

Python

```
student = {  
    "name": "Rohit",  
    "age": 21,  
    "course": "Python"  
}
```

Why is it called Mapping Type?

Because:

- One value maps to another
- Keys are linked to values

Example:

Python

```
"name" → "Rohit"  
"age" → 21
```

Memory Allocation:

- Uses Hash Table internally
- Fast searching mechanism
- Key-based memory access

Characteristics:

- Mutable
- Ordered (Python 3.7+)
- No duplicate keys

Example:

Python

```
student["city"] = "Hyderabad"
```

Mutable or Immutable?

- Mutable

Why?

Dictionary structure allows insertion and modification dynamically.

5. BOOLEAN TYPE

bool – Boolean

Stores only:

```
Python
True
False
```

Used For:

- Conditions
- Decision making
- Comparisons

Example:

```
Python
is_logged_in = True
is_payment_done = False
```

Memory Allocation:

- Internally treated as integers
- True = 1
- False = 0

Mutable or Immutable?

- Immutable

6. NONE TYPE

NoneType

Represents absence of value.

Example:

```
Python
tracking_id = None
```

Used When:

- Value not assigned yet
- Empty state
- Placeholder value

Examples:

```
Python
delivery_partner = None
payment_status = None
```

Memory Allocation:

- Python uses a special singleton object
- Only one None object exists internally

Mutable or Immutable?

- Immutable

MUTABLE vs IMMUTABLE DATATYPES

Mutable Datatypes

Can modify values after creation.

Examples:

```
Python  
list  
set  
dict
```

Explanation:

Memory content can be changed without creating a new object.

Immutable Datatypes

Cannot modify values after creation.

Examples:

```
Python  
int  
float  
str  
tuple  
frozenset  
bool  
NoneType
```

Explanation:

If value changes, Python creates a completely new object in memory.

Mutable Objects

- Same memory object can be modified
- Flexible memory handling

- Used for dynamic data

Example:

```
Python
numbers = [1, 2]
```

Immutable Objects

- New object created after modification
- Safer memory management
- Prevents accidental changes

Example:

```
Python
x = 10
x = 20
```

ORDERED vs UNORDERED DATATYPES

Ordered Datatypes

Elements maintain insertion order.

Examples:

```
Python
list
tuple
str
dict
```

Unordered Datatypes

No fixed position of elements.

Examples:

```
Python
set
frozenset
```

INDEXED vs NON-INDEXED

Indexed Datatypes

Supports position-based access.

Examples:

```
Python
list
tuple
str
```

Non-Indexed Datatypes

Does not support positional access.

Examples:

```
Python
set
frozenset
dict
```

CHECKING DATATYPE USING type()

Example:

```
Python
price = 999.99
items = ["Watch", "Shoes"]

print(type(price))
print(type(items))
```

Output:

```
Python
<class 'float'>
<class 'list'>
```

SUMMARY TABLE

Datatype	Category	Ordered	Mutable	Indexed
int	Numeric	No	No	No
float	Numeric	No	No	No
complex	Numeric	No	No	No
str	Sequence	Yes	No	Yes
list	Sequence	Yes	Yes	Yes
tuple	Sequence	Yes	No	Yes
set	Set	No	Yes	No
frozenset	Set	No	No	No
dict	Mapping	Yes	Yes	Key Based
bool	Boolean	No	No	No
NoneType	Special	No	No	No

TYPE CONVERSIONS

1. int DATATYPE CONVERSIONS

Conversion	Possible?	Example	Output
int → float	Yes	float(10)	10.0
int → complex	Yes	complex(10)	(10+0j)
int → str	Yes	str(10)	"10"
int → bool	Yes	bool(10)	True
int → list	No	list(10)	Error
int → tuple	No	tuple(10)	Error
int → set	No	set(10)	Error
int → frozenset	No	frozenset(10)	Error
int → dict	No	dict(10)	Error

2. float DATATYPE CONVERSIONS

Conversion	Possible?	Example	Output
float → int	Yes	int(10.8)	10
float → complex	Yes	complex(10.8)	(10.8+0j)
float → str	Yes	str(10.8)	"10.8"
float → bool	Yes	bool(10.8)	True
float → list	No	list(10.8)	Error
float → tuple	No	tuple(10.8)	Error
float → set	No	set(10.8)	Error
float → frozenset	No	frozenset(10.8)	Error
float → dict	No	dict(10.8)	Error

3. complex DATATYPE CONVERSIONS

Conversion	Possible?	Example	Output
complex → int	No	int(5+2j)	Error
complex → float	No	float(5+2j)	Error
complex → str	Yes	str(5+2j)	"(5+2j)"
complex → bool	Yes	bool(5+2j)	True
complex → list	No	list(5+2j)	Error
complex → tuple	No	tuple(5+2j)	Error
complex → set	No	set(5+2j)	Error
complex → frozenset	No	frozenset(5+2j)	Error
complex → dict	No	dict(5+2j)	Error

4. str DATATYPE CONVERSIONS

Conversion	Possible?	Example	Output
str → int	Yes	int("100")	100
str → float	Yes	float("55.5")	55.5
str → complex	Yes	complex("5+2j")	(5+2j)
str → str	Yes	str("Python")	"Python"
str → bool	Yes	bool("Python")	True
str → list	Yes	list("Python")	['P','y','t','h','o','n']
str → tuple	Yes	tuple("Python")	('P','y','t','h','o','n')
str → set	Yes	set("Python")	{'P','y','t','h','o','n'}
str → frozenset	Yes	frozenset("Python")	frozenset({'P','y','t','h','o','n'})
str → dict	No	dict("Python")	Error

5. bool DATATYPE CONVERSIONS

Conversion	Possible?	Example	Output
bool → int	Yes	int(True)	1
bool → float	Yes	float(True)	1.0
bool → complex	Yes	complex(True)	(1+0j)
bool → str	Yes	str(True)	"True"
bool → bool	Yes	bool(True)	True
bool → list	No	list(True)	Error
bool → tuple	No	tuple(True)	Error
bool → set	No	set(True)	Error
bool → frozenset	No	frozenset(True)	Error
bool → dict	No	dict(True)	Error

6. list DATATYPE CONVERSIONS

Conversion	Possible?	Example	Output
list → int	No	int([1,2])	Error
list → float	No	float([1,2])	Error
list → complex	No	complex([1,2])	Error
list → str	Yes	str([1,2])	"[1, 2]"
list → bool	Yes	bool([1,2])	True
list → tuple	Yes	tuple([1,2])	(1,2)
list → set	Yes	set([1,2,2])	{1,2}
list → frozenset	Yes	frozenset([1,2])	frozenset({1,2})
list → dict	Yes	dict([("a",1),("b",2)])	{'a':1,'b':2}

7. tuple DATATYPE CONVERSIONS

Conversion	Possible?	Example	Output
tuple → int	No	int((1,2))	Error
tuple → float	No	float((1,2))	Error
tuple → complex	No	complex((1,2))	Error
tuple → str	Yes	str((1,2))	"(1, 2)"
tuple → bool	Yes	bool((1,2))	True
tuple → list	Yes	list((1,2))	[1,2]
tuple → set	Yes	set((1,2,2))	{1,2}
tuple → frozenset	Yes	frozenset((1,2))	frozenset({1,2})
tuple → dict	Yes	dict(((("a",1),("b",2))))	{'a':1,'b':2}

8. set DATATYPE CONVERSIONS

Conversion	Possible?	Example	Output
set → int	No	int({1,2})	Error
set → float	No	float({1,2})	Error
set → complex	No	complex({1,2})	Error
set → str	Yes	str({1,2})	"{1, 2}"
set → bool	Yes	bool({1,2})	True
set → list	Yes	list({1,2})	[1,2]
set → tuple	Yes	tuple({1,2})	(1,2)
set → frozenset	Yes	frozenset({1,2})	frozenset({1,2})
set → dict	Yes	dict({("a",1),("b",2)})	{'a':1,'b':2}

9. frozenset DATATYPE CONVERSIONS

Conversion	Possible ?	Example	Output
frozenset → int	No	int(frozenset({1,2}))	Error
frozenset → float	No	float(frozenset({1,2}))	Error
frozenset → complex	No	complex(frozenset({1,2}))	Error
frozenset → str	Yes	str(frozenset({1,2}))	"frozenset({1,2})"
frozenset → bool	Yes	bool(frozenset({1,2}))	True
frozenset → list	Yes	list(frozenset({1,2}))	[1,2]
frozenset → tuple	Yes	tuple(frozenset({1,2}))	(1,2)
frozenset → set	Yes	set(frozenset({1,2}))	{1,2}
frozenset → dict	Yes	dict(frozenset({"a",1}, {"b",2}))	{'a':1,'b':2}

10. dict DATATYPE CONVERSIONS

Conversion	Possible?	Example	Output
dict → int	No	int({"a":1})	Error
dict → float	No	float({"a":1})	Error
dict → complex	No	complex({"a":1})	Error
dict → str	Yes	str({"a":1})	"{'a':1}"
dict → bool	Yes	bool({"a":1})	True
dict → list	Yes	list({"a":1,"b":2})	['a','b']
dict → tuple	Yes	tuple({"a":1,"b":2})	('a','b')
dict → set	Yes	set({"a":1,"b":2})	{'a','b'}
dict → frozenset	Yes	frozenset({"a":1,"b":2})	frozenset({'a','b'})
dict → dict	Yes	dict({"a":1})	{'a':1}

BOOLEAN RULES

Value	bool() Output
bool(0)	False
bool(0.0)	False
bool("")	False
bool([])	False
bool(())	False
bool(set())	False
bool({})	False
bool(None)	False
Non-empty values	True